Java Interview Questions - Set 2

11. What is bytecode in Java?

Bytecode is the intermediate code that javac produces when it compiles Java source. It's the language the JVM actually runs, sitting between human-readable Java and CPU-specific machine code.

```
Source code (.java)
       |
       v
     javac
       |
       v
Bytecode (.class)
       |
       v
JVM (interpreter + JIT)
       |
       v
Native machine code
```

Key facts:

- Stored in `.class` files, one per top-level class.
- Each file starts with the magic number `0xCAFEBABE` (4 bytes).
- Platform-independent. The same `.class` file runs on any JVM regardless of OS or CPU.
- Compact and stack-based. Operations push and pop values from an operand stack.
- Inspect with `javap -c YourClass.class`.

Example. Source:

```java
public int add(int a, int b) {
    return a + b;
}
```

Bytecode (via `javap`):

```
public int add(int, int);
  Code:
     0: iload_1     // push parameter a
     1: iload_2     // push parameter b
     2: iadd        // pop two ints, add, push result
     3: ireturn     // return the top of the stack
```

The JVM runs this bytecode by interpreting it or compiling it to native code with the JIT. Bytecode is what makes Java's "write once, run anywhere" promise possible.

12. Difference between this() and super() in Java?

Both are calls to other constructors. They differ in which constructor they call.

Aspect	`this()`	`super()`
Calls	Another constructor in the same class	A constructor of the parent class
Used for	Constructor chaining within the class	Initializing inherited state
Position	Must be the first statement	Must be the first statement
Both in one constructor?	No, you can use only one	No, you can use only one
Default behavior	Not added by compiler	Compiler auto-inserts `super()` if you don't

Example with both in a class hierarchy:

```
public class Animal {
    protected String name;
    public Animal(String name) {
        this.name = name;
    }
}

public class Dog extends Animal {
    private String breed;

    public Dog(String name) {
        this(name, "unknown"); // calls Dog's two-arg constructor (same class)
    }

    public Dog(String name, String breed) {
        super(name); // calls Animal's constructor (parent class)
        this.breed = breed;
    }
}
```

What happens when you call `new Dog("rex")`:

1. `Dog(String name)` runs, calls `this(name, "unknown")`.
2. `Dog(String, String)` runs, calls `super(name)`.
3. `Animal(String)` runs, sets `this.name`.
4. Control returns to `Dog(String, String)`, sets `this.breed`.
5. Control returns to `Dog(String)`, which finishes.

Rule of thumb: `this()` is horizontal (within the class). `super()` is vertical (up the inheritance chain).

13. What is a class?

A class is a blueprint for creating objects. It defines the data (fields) and behavior (methods) that every object of that type will have.

```
public class Car {
    // fields (state)
    private String model;
    private int year;
    private double mileage;

    // constructor
    public Car(String model, int year) {
        this.model = model;
        this.year = year;
        this.mileage = 0;
    }

    // methods (behavior)
    public void drive(double km) {
        mileage += km;
    }

    public double getMileage() {
        return mileage;
    }
}
```

A class itself isn't an object. It's the template. You use it to create objects:

```
Car myCar = new Car("Tesla Model 3", 2024);
myCar.drive(50);
```

What a class can contain:

- Fields: the data each object holds.
- Constructors: how new instances get built.
- Methods: the behavior the objects expose.
- Static members: data and methods that belong to the class itself, not instances.
- Inner classes: classes defined inside another class.
- Initializer blocks: code that runs during construction.

Classes can extend exactly one parent class (`extends`) and implement many interfaces (`implements`).

14. What is an object?

An object is an instance of a class. It's the actual thing that lives in memory at runtime, holding state and responding to method calls.

```
Car myCar = new Car("Tesla Model 3", 2024);
```

Three things happen on that line:

1. `new Car(...)` allocates memory on the heap for a `Car` object.
2. The constructor runs, setting up the initial state.
3. The reference to that memory location is assigned to `myCar`.

Objects have:

- State: values of the fields at any moment (`model = "Tesla Model 3"`, `year = 2024`, `mileage = 0`).
- Behavior: the methods you can call on them (`drive`, `getMileage`).
- Identity: even two objects with the same state are different objects in memory. `==` distinguishes them.

```
Car a = new Car("Civic", 2020);
Car b = new Car("Civic", 2020);
a == b         // false, different objects
a.equals(b)    // depends on whether Car overrides equals()
```

Every Java object is an instance of `Object` (the root class), even if you don't extend anything explicitly. That's where `equals`, `hashCode`, `toString`, and `getClass` come from.

15. What is a method in Java?

A method is a named, reusable block of code that lives inside a class and performs some action. Methods take inputs (parameters), optionally return a value, and may have side effects.

Anatomy of a method:

```
public int add(int a, int b) {
    return a + b;
}
```

Part	Example	Meaning
Access modifier	`public`	Who can call it (`public`, `protected`, package-private, `private`)
Other modifiers	`static`, `final`, `abstract`, `synchronized`	Behavior flags
Return type	`int`	What kind of value it returns (`void` for nothing)
Method name	`add`	What you call it
Parameter list	`(int a, int b)`	What it takes as input
Body	`{ return a + b; }`	What it does

Variations:

- Instance method: runs on an object. Has access to `this`.
- Static method: belongs to the class. No `this`. Called as `ClassName.method(...)`.
- Abstract method: declared with no body. Subclasses must implement it.
- Final method: can't be overridden.
- Synchronized method: acquires the object's lock before running.
- Native method: implemented in C/C++ via JNI. No body in Java.

A method can throw checked exceptions, which it must declare with `throws`:

```
public String readFile(String path) throws IOException {
    return Files.readString(Path.of(path));
}
```

16. What is encapsulation?

Covered in detail in the dedicated cheat sheet `java-oop-principles.pdf`. Skipped here to avoid duplication.

17. Why is the main() method public, static, and void in Java?

Each modifier serves a specific purpose for the JVM's entry point.

public

The JVM (which lives outside your code) needs to call `main` from another class. Anything less than `public` would hide the method from the JVM.

static

When the JVM starts a program, no objects exist yet. A `static` method belongs to the class itself, so the JVM can invoke it without instantiating anything. If `main` were an instance method, the JVM wouldn't know which constructor to call to build the instance first.

void

`main` doesn't return a value to anything meaningful. The JVM doesn't expect a return. Process exit codes come from `System.exit(int)` or from uncaught exceptions. `main`'s return value isn't part of the protocol.

Full signature:

```
public static void main(String[] args) {
    // entry point
}
```

`String[] args` carries command-line arguments. Running `java MyApp foo bar` passes `["foo", "bar"]` into `main`.

Recent Java versions added unnamed instance `main` methods, which relax these rules for simple programs:

```
void main() {
    System.out.println("hello");
}
```

For production code in current LTS releases, stick with the classic signature.

18. Explain the main() method in Java

`main` is the entry point of every standalone Java program. The JVM looks for it after loading the class you run, and calls it to start execution.

Standard signature:

```
public static void main(String[] args) {
    // your code
}
```

Variations the JVM accepts:

```
public static void main(String[] args)        // canonical
public static void main(String... args)       // varargs, equivalent
public static final void main(String[] args)  // final is allowed (rare)
static public void main(String[] args)        // modifier order can swap
```

The JVM rejects:

- Missing `public` (JVM can't see it).
- Missing `static` (JVM has no instance to call it on).
- Wrong return type (must be `void`).
- Wrong parameter type (must be `String[]` or `String...`).
- Different name (must be exactly `main`).

What happens when you run `java MyApp foo bar`:

1. The JVM loads the `MyApp` class.
2. Static initializers in `MyApp` run.
3. The JVM looks for `public static void main(String[] args)`.
4. The JVM calls it with `args = ["foo", "bar"]`.
5. The program runs until `main` returns or an uncaught exception escapes it.
6. The JVM exits when `main` ends and no non-daemon threads remain.

A class can have multiple methods named `main` (overloading), but only the canonical one is the entry point. The others are just regular methods.

Recent Java versions allow unnamed instance `main` methods for simpler scripts:

```
void main() {
    System.out.println("hello");
}
```

Saves boilerplate for small programs. Production code still uses the classic form.

19. What is a constructor in Java?

A constructor is a special method that runs when you create a new object with `new`. Its job is to set up the object's initial state.

```
public class User {
    private String name;
    private int age;

    // constructor
    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

User u = new User("alice", 30);
```

Rules:

- Same name as the class.
- No return type (not even `void`).
- Called automatically when you use `new`.
- Can be overloaded (multiple constructors with different parameter lists).
- Can call other constructors with `this(...)` or `super(...)`.

If you don't write any constructor, the compiler adds a default no-arg constructor that takes no arguments and does nothing. The moment you declare any constructor, the compiler stops adding the default.

```
public class Item {
    // no constructor declared
}
Item i = new Item(); // works, compiler-inserted default constructor
```

```
public class Item {
    public Item(String name) {
        // ...
    }
}
Item i = new Item();          // compile error, no no-arg constructor
Item j = new Item("widget");  // works
```

Special types:

- Default constructor: no-arg, compiler-added when no constructor is declared.
- No-arg constructor: any constructor with no parameters, whether default or explicit.
- Parameterized constructor: takes one or more arguments.
- Copy constructor: takes another instance of the same class. Not built into Java, but a common pattern.

```
public class User {
    private String name;
    private int age;

    public User(User other) { // copy constructor
        this.name = other.name;
        this.age = other.age;
    }
}
```

Private constructors block instantiation from outside. Useful for singletons and utility classes.

20. Difference between length and length() in Java?

Both give you a size, but they apply to different types.

Aspect	`length`	`length()`
Type	Field (final, public)	Method
Used on	Arrays	`String`, `StringBuilder`, `StringBuffer`
Syntax	`arr.length`	`s.length()`
Returns	`int`	`int`

Examples:

```
// Arrays use length (field, no parentheses)
int[] numbers = {1, 2, 3, 4, 5};
System.out.println(numbers.length); // 5

String[] names = {"alice", "bob"};
System.out.println(names.length); // 2

// String uses length() (method, with parentheses)
String s = "hello";
System.out.println(s.length()); // 5

StringBuilder sb = new StringBuilder("world");
System.out.println(sb.length()); // 5
```

Why the difference? Arrays are special types built into the JVM. They aren't regular classes, so they expose a final field directly. `String` is a regular class, so it uses a method to return its character count.

Bonus: `List` and other collections use `size()`. Easy to mix up. Quick rule:

- Array: `.length`
- String: `.length()`
- Collection: `.size()`

Quick reference

Question	One-line answer
Bytecode	Intermediate code in `.class` files, run by the JVM
`this()` vs `super()`	Same-class constructor call vs parent-class constructor call
Class	Blueprint defining fields and methods for objects
Object	Instance of a class, lives on the heap
Method	Named block of code that performs an action, lives in a class
Encapsulation	See `java-oop-principles.pdf`
Why `main` is public static void	JVM needs to call it externally without an instance and without a return
`main` method	Entry point of a Java program. Signature is `public static void main(String[] args)`
Constructor	Special method that initializes a new object, same name as the class
`length` vs `length()`	Field on arrays vs method on `String` and builders